DEPARTMENT OF SOFTWARE ENGINEERING
BACHELORS IN SOFTWARE ENGINEERING

Course Code: CT-317 | Course Title: Machine Learning
SE Batch 2023, Fall Semester 2026

Complex Engineering Problem

Term Project Report

Q1. Fake-News Headline Classification with k-Nearest Neighbours
Q2. Sentiment (Emotion) Analysis from Noisy Audio Recordings
Q3. Surface Defect Detection, Classification and Localisation

Group Members

Student No.	**Name**	**Roll No.**
S1	Sumayyah Mohammedy	SE-23013
S2	Amna Khan	SE-23039
S3	Mariha Mahmood	SE-23047
S4	Haris Bin Nasir	SE-23084

Source code: `https://github.com/hbnasir23/ml-cep-ct317`

Submitted to: _____

Date of submission: _____

Abstract

This report covers the three machine-learning problems in the CT-317 Complex Engineering Problem. **Q1** classifies 3,266 news headlines as real or fake with scikit-learn's k-nearest-neighbour classifier on bag-of-words features. With the default Euclidean distance the selected model ($k = 12$) reaches 63.9% test accuracy. Switching to cosine distance raises this to **75.7%**, because cosine distance ignores headline length and compares only which words are used. **Q2** recognises six emotions in 7,442 noisy speech clips (CREMA-D). Clips are low-pass filtered at 4 kHz, and 242 hand-crafted features are extracted (FFT bands, MFCCs, mel-spectrogram, chromagram, spectral shape, prosody and bicoherence). Five classifiers are evaluated on a *speaker-independent* split. The best, an RBF-SVM, reaches **52.4%** accuracy on unseen speakers (chance is 16.7%). Per-speaker feature normalisation raises this to **61.3%**. **Q3** detects holes and pen marks in 1,500 high-resolution images of a paper reel. The pipeline flattens uneven illumination, proposes dark candidate regions and describes them with colour histograms, spatial binning, HOG, LBP, GLCM and shape descriptors. Four classifiers reach **97–99%** test accuracy. A region classifier trained with hard negatives scores every proposal to localise the defect, and the chosen box lies on the true defect in all inspected test images. All code is modular and reproducible (fixed seeds, cached features). Each question can be run with a single command. As an innovation, an *explainable* web application lets users test all three models with their own input (typed headlines, live microphone recordings, uploaded images) and shows *why* each model decided as it did. The code is public at `https://github.com/hbnasir23/ml-cep-ct317`.

Contents

1 Introduction **1**
- 1.1 Objectives . . . 1
- 1.2 Tools and environment . . . 1
- 1.3 General methodology . . . 1
- 1.4 Repository structure . . . 1

2 Q1: Classification with Nearest Neighbours **3**
- 2.1 Problem statement . . . 3
- 2.2 Dataset . . . 3
- 2.3 Methodology . . . 3
  - 2.3.1 `load_data`: pre-processing and splitting . . . 3
  - 2.3.2 `select_knn_model`: model selection . . . 3
- 2.4 Results . . . 4
- 2.5 Why does the cosine metric perform better? . . . 4
  - 2.5.1 How `metric='cosine'` computes distance . . . 4
  - 2.5.2 The hint dataset . . . 4
  - 2.5.3 Consequences for headlines . . . 5

3 Q2: Sentiment Analysis from Audio Recordings **6**
- 3.1 Problem statement . . . 6
- 3.2 Dataset . . . 6
- 3.3 Data selection and splitting . . . 6
- 3.4 Pre-processing: noise removal . . . 7
- 3.5 Feature extraction . . . 7
  - 3.5.1 Literature review . . . 7
  - 3.5.2 Implemented features . . . 8
- 3.6 Classifiers and training protocol . . . 8
- 3.7 Results . . . 9
- 3.8 Further analysis . . . 10
  - 3.8.1 Feature-group ablation . . . 10
  - 3.8.2 Choice of the low-pass cut-off . . . 10
  - 3.8.3 Ensemble and per-speaker normalisation . . . 10
- 3.9 Limitations . . . 11

4 Q3: Surface Defect Detection and Classification **12**
- 4.1 Problem statement . . . 12
- 4.2 Dataset analysis . . . 12
- 4.3 Pre-processing and candidate regions . . . 12
- 4.4 Feature extraction . . . 13
  - 4.4.1 Literature review . . . 13
  - 4.4.2 Implemented features . . . 14

4.5 Task (i): three-class classification . . . 14
4.6 Task (ii): defect localisation . . . 15
4.6.1 Method . . . 15
4.6.2 Localisation results . . . 16
4.7 Limitations . . . 17

5 Software Design and Ease of Use 18
5.1 Organisation of the code . . . 18
5.2 How to run . . . 18
5.3 Interactive web application . . . 18
5.4 Public repository . . . 18
5.5 Reproducibility . . . 19

6 Innovation: An Explainable Interactive Application 20
6.1 Q1: explain a headline by its neighbours . . . 20
6.2 Q2: live microphone input and "what the filter removed" . . . 20
6.3 Q3: defect localisation with a look inside the pipeline . . . 21
6.4 Methodological innovations . . . 22

7 Team Contribution 23

8 Conclusion and Future Work 24

A Requirement Checklist 27

B Selected Hyper-parameter Grids 28

List of Figures

2.1 Training and validation accuracy of KNN as a function of k. 4

3.1 Effect of the 4 kHz low-pass filter on a noisy clip: waveform (top) and spectrogram (bottom). Energy above 4 kHz, which is mostly noise, is removed. 7
3.2 Validation accuracy, test accuracy and test macro-F1 of the five classifiers. 9
3.3 Row-normalised test confusion matrices of the five classifiers. 9

4.1 Pre-processing for a test image of each class: grayscale input, illumination-flattened z-map, and the patch around the top candidate. 13
4.2 Test accuracy of the four classifiers for each feature set. 15
4.3 Test confusion matrices with all features. 15
4.4 Detected bounding boxes on 24 random defective test images (zoomed on the box; titles show the true class and the prediction; red = wrong class). 17
4.5 All misclassified test images. Each box is still on the defect; the errors are faint holes (predicted paper) and elongated holes resembling pen strokes. 17

6.1 Q1 tab: prediction for user headlines and the 12 nearest training headlines that explain it. 20
6.2 Q2 tab: microphone recording (top); prediction, probabilities and the spectrogram before and after filtering (bottom). 21
6.3 Q3 tab: detected pen mark with bounding box and zoom (top); z-map with the five scored proposals, where #1 (green) is the selected defect (bottom). 22

List of Tables

2.1 Q1 model selection results. 4
2.2 Distances for the hint dataset. 5

3.1 Speaker-independent split of CREMA-D. 6
3.2 Audio feature groups (242 features per clip). 8
3.3 Q2 emotion recognition on unseen speakers (6 classes, chance = 0.167). 9
3.4 Feature ablation (SVM, test accuracy). 10
3.5 Effect of the low-pass cut-off frequency (SVM, test accuracy). 10
3.6 Ensemble and speaker normalisation (test accuracy / macro-F1). 10

4.1 Image feature groups (4,959 features in total). 14
4.2 Q3 three-class test accuracy (225 test images) for each feature set. Validation accuracy in parentheses. 14
4.3 Region classifier (multi-proposal decision) accuracy. 16

7.1 Individual contributions. 23
7.2 Responsibility matrix (**L** = lead, **S** = support). 23

A.1 CEP requirements and where they are addressed. 27

Chapter 1

Introduction

1.1 Objectives

The Complex Engineering Problem (CEP) asks for three complete machine-learning pipelines, one in each of three application areas:

1. **Classification (text).** Learn to use an existing library implementation (scikit-learn's `KNeighborsClassifier`), handle train/validation/test splits, and study the effect of the distance metric.
2. **Natural-language and audio processing.** Select data, design suitable splits, remove noise with low-pass filtering, extract audio features identified in the literature, and compare KNN, Logistic Regression, Naive Bayes, SVM and a Neural Network.
3. **Computer vision.** Design a classical feature pipeline (colour histogram, spatial binning, HOG and further features from the literature), compare SVM, Naive Bayes, Logistic Regression and a Neural Network for three-class defect classification, and localise defects with bounding boxes.

1.2 Tools and environment

All work was done in Python 3.13 on Windows 11 (12 CPU cores) with scikit-learn 1.8 [1], NumPy, SciPy, pandas and matplotlib. `librosa` 1.0 [2] was used for audio analysis, and OpenCV 5.0 and scikit-image 0.26 [3] for image processing. The interactive demo uses Streamlit. Exact package requirements are listed in `requirements.txt`.

1.3 General methodology

Three principles were applied to every question so that the reported numbers are trustworthy:

- **Three-way split.** Hyper-parameters are chosen *only* on a validation set. The test set is used exactly once, to report the final result.
- **No information leakage.** All pre-processing that learns from data (vocabulary, feature scaling) is fitted on the training data only. Splits are designed so that near-duplicate information (for example, the same speaker) cannot appear in both training and test data.
- **Reproducibility.** All random operations use seed 42, and expensive feature extraction is cached on disk so that every table and figure can be regenerated.

1.4 Repository structure

```
ml_cep/
  README.md, requirements.txt, app.py   (Streamlit demo for all questions)
  q1/  q1_knn.py                         data/, results/, models/
  q2/  audio_features.py  q2_audio_sentiment.py  q2_extras.py  predict.py
       dataset_repo/ (CREMA-D clone), results/, models/, cache/
  q3/  defect_features.py  q3_defect_detection.py  predict.py  error_analysis.py
       data/out/{paper,hole,penMark}/, results/, models/, cache/
  report/  report.tex, figures/
```

Chapter 2

Q1: Classification with Nearest Neighbours

2.1 Problem statement

The task is to classify news headlines as *real* or *fake* with scikit-learn's `KNeighborsClassifier`. We must (a) load and vectorise the data and split it 70/15/15, (b) select $k \in \{1, \ldots, 20\}$ by validation accuracy with the default metric, and (c) repeat the selection with `metric='cosine'` and explain the difference.

2.2 Dataset

The cleaned dataset in `hw1_data.zip` contains 1,968 real headlines (`clean_real.txt`, from the "million headlines" corpus, restricted to headlines containing "trump" after October 2016) and 1,298 fake headlines (`clean_fake.txt`, from the Kaggle fake-news dataset), 3,266 in total. The CSV files in the same archive contain an unrelated regression dataset and are not used.

2.3 Methodology

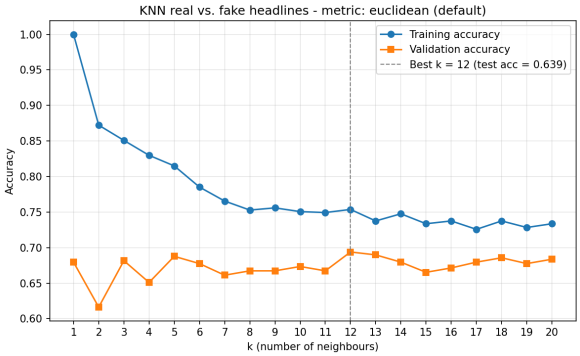
2.3.1 `load_data`: pre-processing and splitting

1. Read both files and label them `real`=1 and `fake`=0.
2. Make a *stratified* random split into 70% training, 15% validation and 15% test data (2,286 / 490 / 490 headlines). Stratification keeps the real/fake ratio (60/40) the same in every subset.
3. Fit a `CountVectorizer` on the *training headlines only*, giving a vocabulary of 4,804 words, and transform all three subsets. Each headline becomes a sparse vector of word counts. Words that appear only in validation or test headlines are ignored, exactly as they would be for genuinely new data.

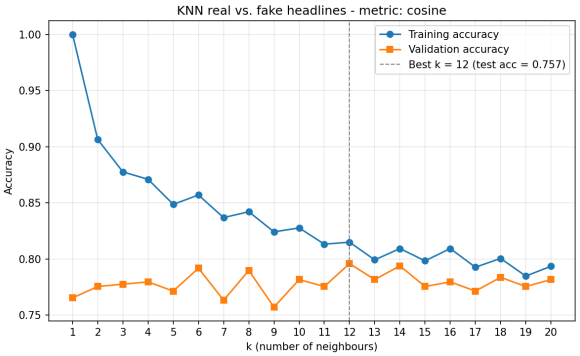
2.3.2 `select_knn_model`: model selection

For every $k = 1, \ldots, 20$, a `KNeighborsClassifier(n_neighbors=k)` is trained with all other arguments at their defaults (Minkowski distance with $p = 2$, i.e. Euclidean; uniform weights). Training and validation accuracy are recorded. The k with the highest validation accuracy is selected, with ties broken towards the larger (smoother) k. The selected model's accuracy on the test set is then reported. Part (c) repeats the procedure with `metric='cosine'`.

2.4 Results



(a) Euclidean (default) metric



(b) Cosine metric

Figure 2.1: Training and validation accuracy of KNN as a function of k.

Table 2.1: Q1 model selection results.

Metric	Best k	Train acc.	Validation acc.	**Test acc.**
Euclidean (default)	12	0.754	0.694	0.639
Cosine	12	0.815	0.796	**0.757**

Observations. With $k = 1$ the training accuracy is 100%: every training headline is its own nearest neighbour, so the model memorises the data and over-fits (validation accuracy is only 0.680 for Euclidean and 0.765 for cosine). As k grows, training accuracy falls and the gap to validation accuracy narrows, which is the expected bias–variance trade-off. Cosine distance is better than Euclidean for *every* k (Figure 2.1), by about 10 percentage points on validation data and 11.8 points on the test set.

2.5 Why does the cosine metric perform better?

2.5.1 How `metric='cosine'` computes distance

For two count vectors $\mathbf{u}, \mathbf{v}$,

$$d_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = 1 - \cos\theta, \tag{2.1}$$

where θ is the angle between the vectors. The distance depends only on the *direction* of the vectors, i.e. on the relative proportions of the words, not on their length. It is 0 for headlines with identical word proportions and 1 for headlines with no word in common. The Euclidean distance $d_E = \|\mathbf{u} - \mathbf{v}\|_2$ depends on both direction and magnitude.

2.5.2 The hint dataset

For `['cat', 'bulldozer', 'cat cat cat']` the count vectors over the vocabulary (*bulldozer*, *cat*) are $(0, 1)$, $(1, 0)$ and $(0, 3)$.

Table 2.2: Distances for the hint dataset.

Pair	Euclidean	Cosine
cat – bulldozer	$\sqrt{2} = 1.414$	1
cat – cat cat cat	2.000	**0**
bulldozer – cat cat cat	3.162	1

Under Euclidean distance, *cat* is closer to *bulldozer* than to *cat cat cat*, even though the latter is about the same thing. Cosine distance gives the intuitive answer.

2.5.3 Consequences for headlines

- Headlines have different lengths and some repeat words. Under Euclidean distance, short headlines are close to *all* other short headlines regardless of topic (their vectors are all near the origin), and long headlines are far from everything. The neighbours are therefore chosen largely by length, which carries little information about real versus fake.
- Cosine distance normalises away the length and measures word *overlap*. The nearest neighbours then share vocabulary (topic, named entities, sensational words typical of fake headlines), which is what separates the two classes.
- Bag-of-words vectors are very sparse and high-dimensional (4,804 dimensions, roughly 8 non-zeros each). In such spaces Euclidean distances concentrate and become less discriminative, whereas cosine similarity remains a meaningful measure of shared content. This is why it is the standard choice in information retrieval [4].

Chapter 3

Q2: Sentiment Analysis from Audio Recordings

3.1 Problem statement

The goal is to build a classifier that identifies the emotion expressed in a recorded speech clip. The audio must first be de-noised with low-pass filtering. Features identified in the literature are then extracted, and five classifiers are compared: KNN, Logistic Regression, Naive Bayes, SVM and a Neural Network.

3.2 Dataset

The provided repository (a noisy version of CREMA-D [5]) contains **7,442** WAV clips (16 kHz, mono, 16-bit, about 2–3 s each) recorded by **91 actors** (48 male, 43 female, aged 20–74, of diverse ethnicities). Each actor reads 12 sentences with one of six emotions. Three of the four emotion-intensity levels (low, medium, high) are used only for one sentence; the rest are "unspecified". File names encode the metadata, e.g. `1001_DFA_ANG_XX.wav` = actor 1001, sentence DFA, *Anger*, unspecified intensity. The classes are almost balanced: 1,271 clips each for Anger, Disgust, Fear, Happy and Sad, and 1,087 for Neutral.

3.3 Data selection and splitting

The emotion classifier must work on speakers it has never heard. If clips were split at random, every actor would appear in both training and test data, and the classifier could partly recognise the *voice* instead of the *emotion*, giving over-optimistic results. We therefore use a **speaker-independent** split: the 91 actors (not the clips) are divided randomly 70/15/15 with `GroupShuffleSplit`. The code asserts that the three actor sets are disjoint. All clips are used, all six emotions are present in every split in the original proportions, and all 12 sentences appear in every split (Table 3.1).

Table 3.1: Speaker-independent split of CREMA-D.

Split	Actors	ANG	DIS	FEA	HAP	NEU	SAD	Total
Train	63	880	880	880	880	752	880	5,152
Validation	14	195	195	195	195	167	195	1,142
Test	14	196	196	196	196	168	196	1,148

3.4 Pre-processing: noise removal

1. **Low-pass filtering.** A 6th-order Butterworth low-pass filter with a cut-off of **4 kHz** is applied in second-order sections with `sosfiltfilt` (forward–backward). The result therefore has zero phase distortion, so the timing of pitch and energy contours is preserved. Speech intelligibility and most prosodic and emotional cues lie below 4 kHz [6], while the added broadband noise extends up to the 8 kHz Nyquist frequency, so the filter removes about half of the noise band while keeping the speech.
2. **Silence trimming** of leading and trailing segments more than 30 dB below the peak, so that statistics are computed over speech only.
3. **Peak normalisation**, so that loudness differences caused by recording gain do not dominate.

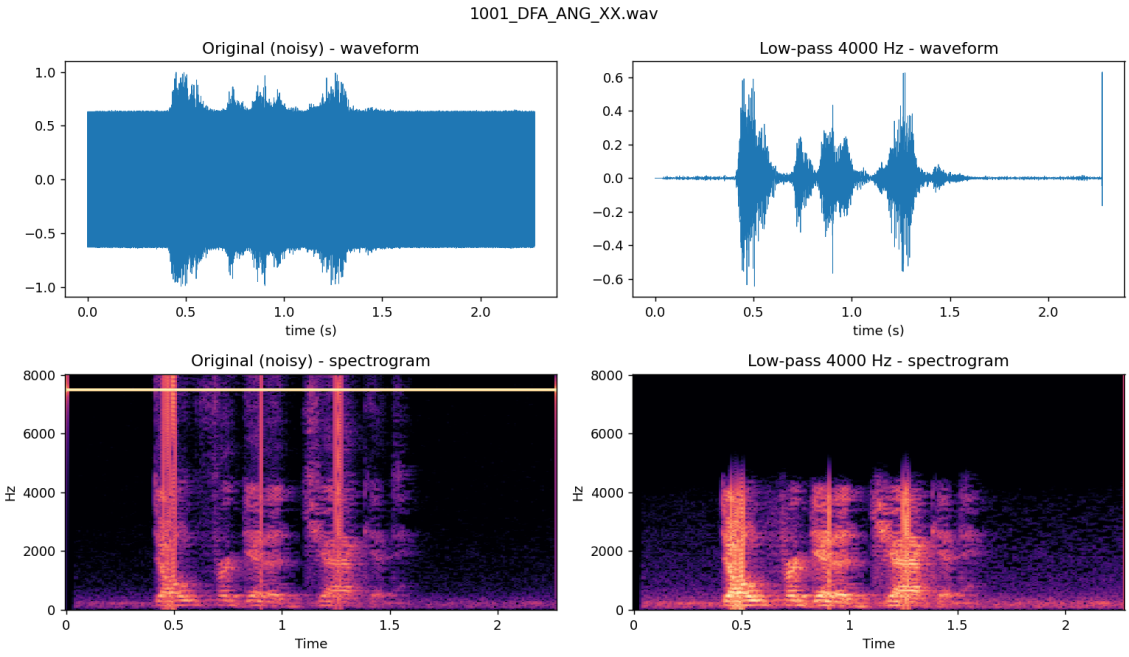


Figure 3.1: Effect of the 4 kHz low-pass filter on a noisy clip: waveform (top) and spectrogram (bottom). Energy above 4 kHz, which is mostly noise, is removed.

3.5 Feature extraction

3.5.1 Literature review

Speech emotion recognition traditionally relies on three families of hand-crafted descriptors [7, 8]:

- *spectral and cepstral* features describing vocal-tract shape and voice quality: MFCCs, mel-spectrogram, spectral centroid, bandwidth, contrast, roll-off and flatness;
- *prosodic* features (fundamental frequency F0, energy, speaking rate), which capture arousal; anger and happiness, for instance, show higher and more variable pitch and energy than sadness;
- *voice-quality and non-linear* features such as higher-order spectra. Bicoherence, the normalised bispectrum, measures quadratic phase coupling between frequency components and has been used for synthetic-speech detection and emotion analysis [9].

The chromagram (12 pitch classes) suggested in the handout is mainly a music feature but carries some intonation information, so it is included and evaluated.

3.5.2 Implemented features

Frame-level features use a 512-sample (32 ms) window and a 160-sample (10 ms) hop. They are summarised over time by their mean and standard deviation, so every clip yields a fixed-length vector of **242** values (Table 3.2).

Table 3.2: Audio feature groups (242 features per clip).

Group	Dim.	Description
Fourier transform	16	log relative energy of the magnitude FFT in 16 equal bands from 0–4 kHz
MFCC	80	20 MFCCs and their deltas (mean, std)
Mel-spectrogram	80	40 log-mel bands up to 4 kHz (mean, std)
Chromagram	24	12 pitch classes (mean, std)
Spectral shape	22	centroid, bandwidth, roll-off, flatness, 7-band spectral contrast (mean, std)
Prosody	12	F0 (YIN) mean, std, 10th/90th percentile and range over voiced frames; RMS energy mean, std and max; zero-crossing rate mean and std; voiced ratio; duration
Bicoherence	8	mean, variance, skewness, kurtosis of magnitude and phase of the segment-averaged bicoherence (256-sample segments, 64 frequency bins)

The bicoherence is estimated as

$$b(f_1, f_2) = \frac{|\mathbb{E}\left[X(f_1)X(f_2)X^*(f_1+f_2)\right]|}{\sqrt{\mathbb{E}\left[|X(f_1)X(f_2)|^2\right]\,\mathbb{E}\left[|X(f_1+f_2)|^2\right]}}, \tag{3.1}$$

where the expectation is an average over Hann-windowed segments. Extraction of all 7,442 clips runs in parallel and takes about one minute.

3.6 Classifiers and training protocol

Each classifier is wrapped in a pipeline with a `StandardScaler`, which is fitted on the training data only:

- **KNN**: $k \in \{3, \ldots, 31\}$, uniform or distance weights, Euclidean or cosine distance.
- **Logistic Regression** (multinomial, L2): $C \in \{0.001, \ldots, 10\}$.
- **Gaussian Naive Bayes**: variance smoothing 10^{-9} to 10^{-1}.
- **SVM** with RBF kernel: $C \in \{0.3, \ldots, 30\}$, $\gamma \in \{\text{scale}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}\}$.
- **Neural Network** (MLP, ReLU, Adam, early stopping): hidden layers (128), (256), (256,128), (512,256); L2 penalty $\alpha \in \{10^{-3}, \ldots, 1\}$.

All grid points are trained on the training speakers and scored on the validation speakers. The best setting is refit on training plus validation data and evaluated *once* on the 14 unseen test speakers.

3.7 Results

Table 3.3: Q2 emotion recognition on unseen speakers (6 classes, chance = 0.167).

Model	Selected hyper-parameters	Val. acc.	**Test acc.**	Test macro-F1
KNN	$k = 15$, cosine, uniform	0.469	0.408	0.381
Logistic Regression	$C = 1$	0.525	0.519	0.519
Naive Bayes	smoothing 10^{-9}	0.387	0.402	0.354
SVM (RBF)	$C = 3$, γ=scale	0.531	**0.524**	**0.522**
Neural Network	(128), $\alpha = 0.1$	**0.537**	0.513	0.513

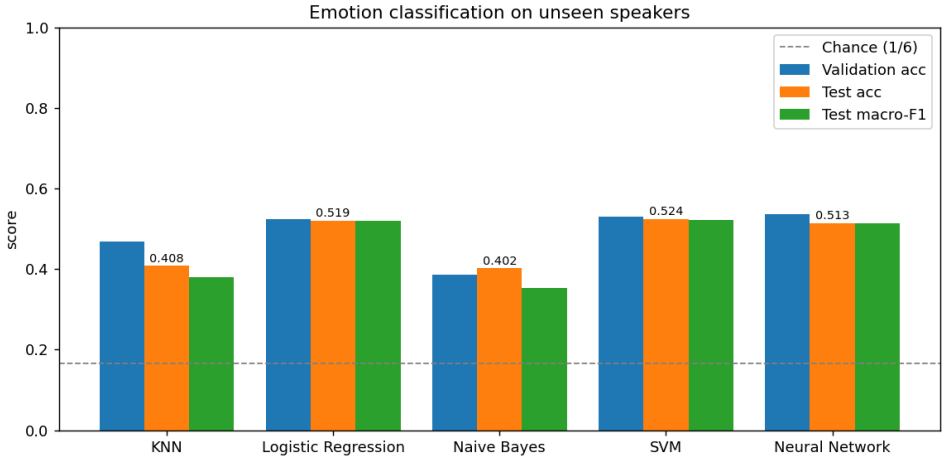


Figure 3.2: Validation accuracy, test accuracy and test macro-F1 of the five classifiers.

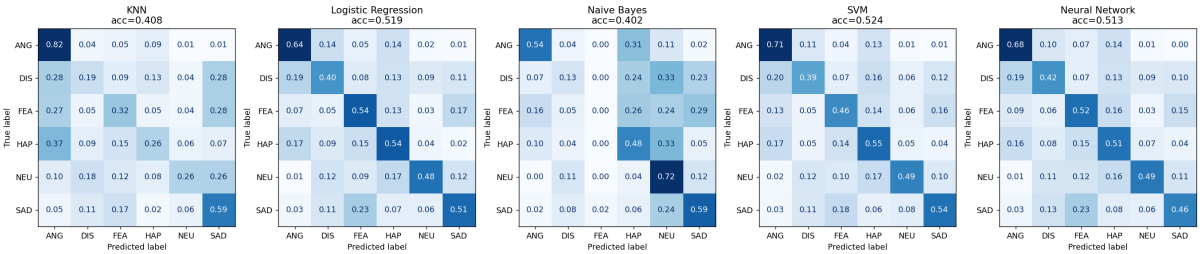


Figure 3.3: Row-normalised test confusion matrices of the five classifiers.

Discussion. The discriminative models (SVM, Logistic Regression, MLP) perform similarly at 51–52%, more than three times chance. This is in line with published speaker-independent CREMA-D results that use hand-crafted features [5]; human raters reach only about 63% on audio alone. Naive Bayes is weakest, because its independence assumption is strongly violated (MFCC, mel and spectral features are highly correlated). KNN suffers from the high dimensionality and from speaker differences dominating the distances. Per class (SVM), *Anger* is recognised best (F1 = 0.63, recall 0.71) and *Disgust* worst (F1 = 0.44). The main confusions in Figure 3.3 are Disgust↔Anger, Fear↔Sad and Happy↔Anger. These are emotions that share arousal (Anger and Happy are both high) or valence, which acoustic features capture poorly. Although the MLP had the highest validation accuracy and is therefore saved as the deployed model, the SVM generalised slightly better to the test speakers. The difference (1.1 points, about 13 clips) is within the variation expected from 14 test speakers.

3.8 Further analysis

3.8.1 Feature-group ablation

Using the selected SVM, each feature group was evaluated alone, and the full set was evaluated with each group left out (Table 3.4).

Table 3.4: Feature ablation (SVM, test accuracy).

Group	Dim.	Group alone	All *minus* group
FFT bands	16	0.383	0.525
MFCC	80	**0.490**	**0.504**
Mel-spectrogram	80	0.456	0.527
Chromagram	24	0.351	0.525
Spectral shape	22	0.419	0.520
Prosody	12	0.449	0.515
Bicoherence	8	0.289	0.524
All features	242	0.524	

MFCCs are the most informative group: they are the best on their own, and removing them causes the largest drop. With only 12 values, prosody is remarkably strong (0.449), which confirms that pitch and energy carry emotional arousal. Bicoherence and chroma are weak on their own and redundant given the other groups.

3.8.2 Choice of the low-pass cut-off

Table 3.5: Effect of the low-pass cut-off frequency (SVM, test accuracy).

Cut-off	2 kHz	3 kHz	4 kHz	6 kHz	none (≈8 kHz)
Test accuracy	0.526	0.531	0.524	0.529	0.529

Accuracy hardly changes between 2 kHz and no filtering, because the features are computed in the 0–4 kHz band. The emotional information is concentrated in the low frequencies, so the filter removes noise without losing useful signal. 4 kHz is kept as a conservative choice that preserves the full telephone speech band.

3.8.3 Ensemble and per-speaker normalisation

Table 3.6: Ensemble and speaker normalisation (test accuracy / macro-F1).

Model	Global scaling	Per-speaker normalisation
SVM	0.524 / 0.522	**0.613** / 0.611
Logistic Regression	0.519 / 0.519	0.609 / 0.608
Neural Network	0.513 / 0.513	0.589 / 0.589
Soft-voting ensemble (SVM+LR+NN)	0.538 / 0.537	**0.613** / **0.612**

A soft-voting ensemble of the three strongest models improves the result to 53.8%. A much larger gain comes from **per-speaker normalisation**: every feature is z-scored using the mean and standard deviation of all clips of the *same speaker*. This removes speaker-specific characteristics

(average pitch, timbre, recording level) and keeps the emotion-related deviations, raising test accuracy by about 9 points to **61.3%**. The technique needs several recordings of a speaker but *no labels*, so it is legitimate in practice, e.g. for a call-centre agent or a returning user. Accuracy also depends on emotion intensity: clips with low intensity are hardest (35.7%), and medium and high intensity reach 51.4%.

3.9 Limitations

Accuracy is limited by acted, short and noisy utterances and by genuine ambiguity between emotions. Even human raters disagree on many CREMA-D clips. Deep representations (CNNs on log-mel spectrograms or pre-trained wav2vec-style encoders) would be the next step beyond the classical features required here.

Chapter 4

Q3: Surface Defect Detection and Classification

4.1 Problem statement

The aim is to detect defects in images of a paper reel. This involves (i) classifying each image into one of three classes (defect-free paper, paper with a hole, paper with a pen mark) using a classical feature pipeline and four classifiers (SVM, Naive Bayes, Logistic Regression, Neural Network), and (ii) localising the defect with a bounding box in defective images.

4.2 Dataset analysis

The dataset (downloaded from the Google Drive folder in the handout) contains **1,500** JPEG images of 2000×2000 pixels: 509 *paper*, 489 *hole* and 502 *penMark*. Analysis of the data showed several properties that shaped the design:

- The three colour channels are identical, so the images are effectively **grayscale**, and a "colour histogram" is an intensity histogram.
- The defects are **tiny**: a hole is a dark speck of about 5–25 px (less than 0.01% of the image) and a pen mark is a thin dark stroke of about 15–120 px. By contrast, the paper texture and a strong **illumination gradient** dominate the image (Figure 4.1, left). Global features of the raw image therefore see mostly lighting and texture.
- A few faint dark blemishes appear at the **same position in every image** (dust on the lens or background). A detector must learn to ignore them.
- Image numbers 0–1499 are shared across the classes and interleaved randomly, so the images are not ordered video frames and a stratified random split is appropriate.

The images are split with stratification into 1,050 training, 225 validation and 225 test images (70/15/15).

4.3 Pre-processing and candidate regions

1. **Down-sampling** to 1000×1000 with area interpolation, which preserves small dark spots.
2. **Illumination flattening.** The background is estimated with a wide Gaussian blur ($\sigma = 20$ px) and subtracted, giving the residual $r = I - G_\sigma * I$.
3. **Robust z-score map** $z = (r - \text{median}(r))/(1.4826\,\text{MAD}(r))$. Clean paper texture stays within about $[-7, +10]$, while holes and ink reach -10 to -50, independently of the lighting.

4. **Candidate proposals.** Pixels with $z < -4$ (after light smoothing) are thresholded, a 7×7 morphological closing joins the broken strokes of a pen mark, and connected components are ranked by their "darkness mass" $\sum(-z)$. The top proposals are passed on.

A calibration over all 1,500 images confirmed that the method separates the classes well: the darkest response on clean paper is $-5.9 \pm 0.6\sigma$, compared with $-30.7 \pm 10.5\sigma$ for holes and $-23.8 \pm 4.3\sigma$ for pen marks. Pen-mark regions are about 4–5 times larger than hole regions.

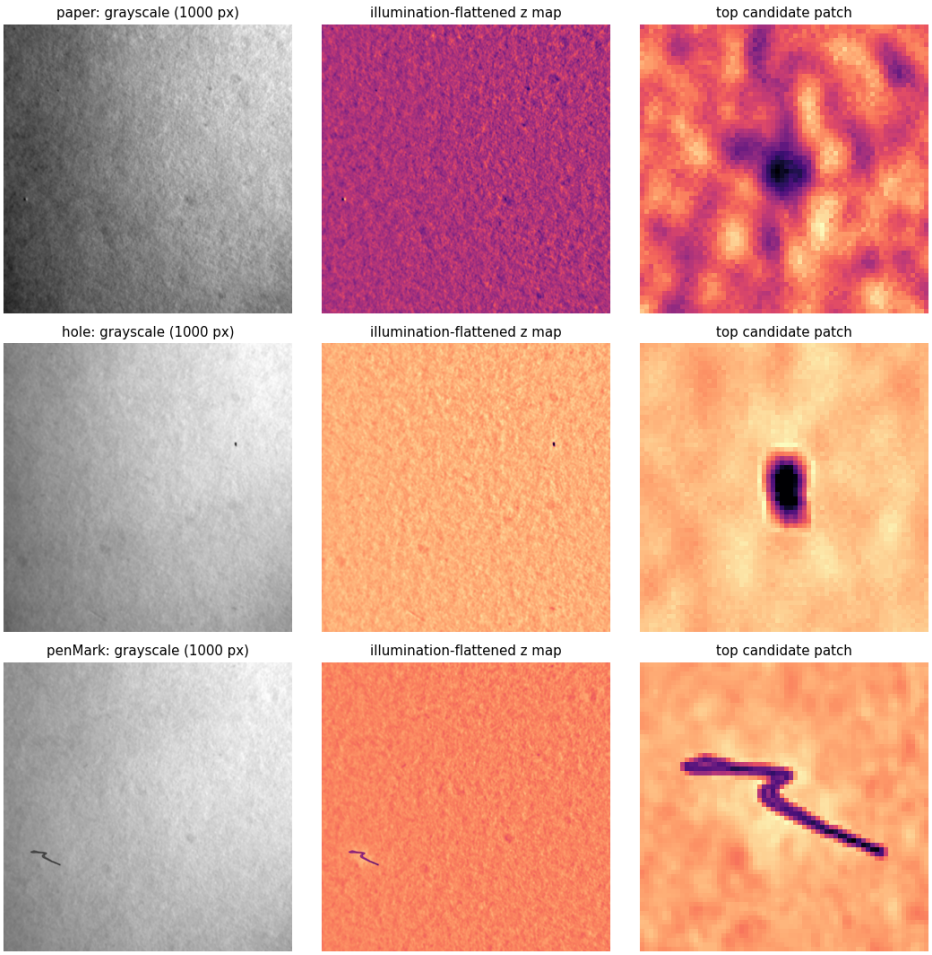


Figure 4.1: Pre-processing for a test image of each class: grayscale input, illumination-flattened z-map, and the patch around the top candidate.

4.4 Feature extraction

4.4.1 Literature review

Classical surface-inspection systems describe defects by intensity statistics, texture and shape [10, 11]. Besides the features in the handout (colour histogram, spatial binning [12], HOG [13]), we used:

- **Local Binary Patterns (LBP)** [14], a grey-scale-invariant texture descriptor widely used for surface and fabric defect detection;
- **Grey-Level Co-occurrence Matrix (GLCM)** statistics [15] (contrast, dissimilarity, homogeneity, energy, correlation), the classical texture features for steel, wood and paper inspection;

- **shape descriptors** of the segmented blob (area, bounding-box aspect ratio, extent, solidity, eccentricity, major and minor axis, circularity, Hu moment invariants [16]), which directly encode "compact spot" versus "elongated stroke".

4.4.2 Implemented features

Table 4.1: Image feature groups (4,959 features in total).

Set	Group	Dim.	Description
Global	Colour histogram	32	intensity histogram of the image
	z histogram	32	log-count histogram of the z-map
	Spatial binning	1,024	z-map min-pooled to 32×32
	HOG	1,764	HOG of the z-map (128×128, 16 px cells, 2×2 blocks, 9 bins)
Patch	z histogram	16	histogram of the candidate patch
	Spatial binning	256	patch resized to 16×16
	HOG	1,764	patch 64×64, 8 px cells, 2×2 blocks, 9 bins
	LBP	10	uniform LBP ($P = 8, R = 1$) histogram
	GLCM	40	5 properties × 2 distances × 4 angles
	Shape	21	blob geometry, darkness statistics, 7 Hu moments, patch scale

The patch is a square crop centred on the candidate, with a side of $\max(64, 1.4\times$ the blob size), resized to 64×64. This keeps the whole pen stroke inside the patch.

Data augmentation. Paper has no preferred orientation, so every training and validation image is also used rotated by 90° and 180° and flipped horizontally (×4 data). Test images are never augmented.

4.5 Task (i): three-class classification

All classifiers use a `StandardScaler`, a validation grid search (SVM: C, γ; Naive Bayes: smoothing; Logistic Regression: C; MLP: architecture and α), refitting on augmented training plus validation data, and a single evaluation on the 225 test images. Three feature sets were compared to measure the value of the candidate patch.

Table 4.2: Q3 three-class test accuracy (225 test images) for each feature set. Validation accuracy in parentheses.

Classifier	Global (2,852)	Patch (2,107)	All (4,959)
SVM (RBF)	0.969 (0.973)	0.982 (**0.991**)	0.987 (0.987)
Naive Bayes	0.942 (0.920)	0.960 (0.960)	0.987 (0.964)
Logistic Regression	0.978 (0.973)	0.978 (0.973)	**0.991** (0.982)
Neural Network (MLP)	0.982 (0.969)	0.973 (0.982)	0.987 (0.978)

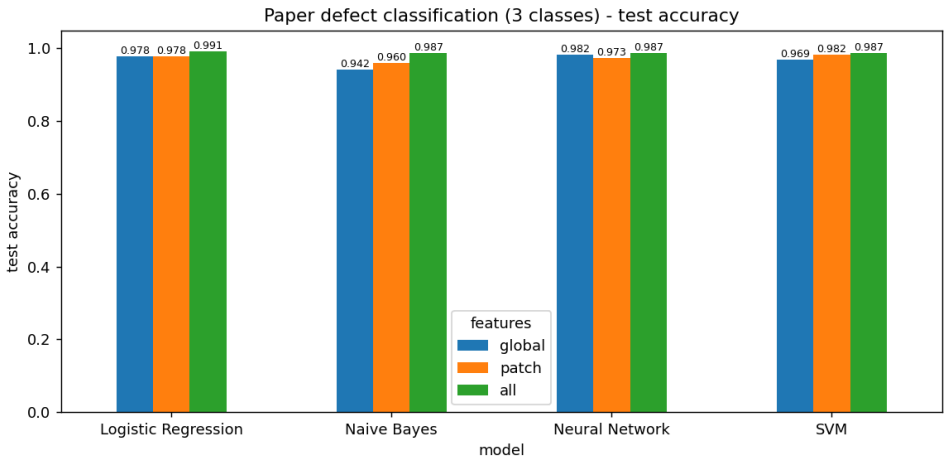


Figure 4.2: Test accuracy of the four classifiers for each feature set.

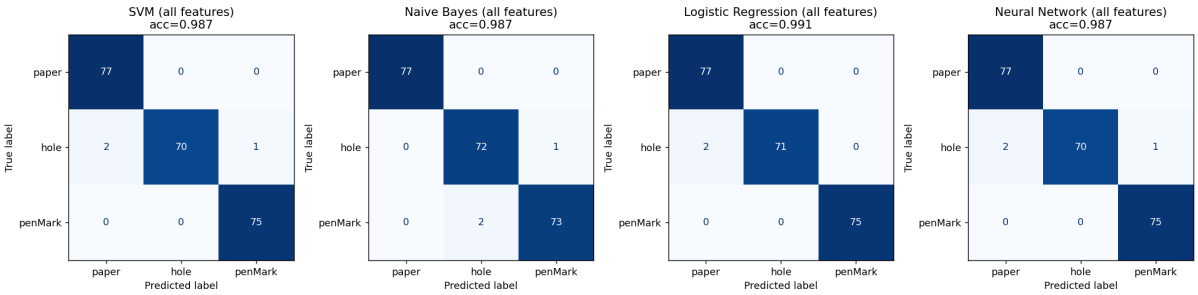


Figure 4.3: Test confusion matrices with all features.

Discussion. All four classifiers perform very well (94–99%) *because* the pre-processing makes the defects visible: on the illumination-flattened z-map, even the global HOG and spatial-binning features carry the defect signal. The candidate-patch features (texture and shape of the isolated object) give the highest validation accuracy, and combining both sets gives the best test accuracy (98.7–99.1%). With all features even Naive Bayes reaches 98.7%, which shows that the shape descriptors make the classes nearly linearly separable. The few remaining errors are almost all holes: very faint or very small holes that look like paper texture, and elongated holes that look like a short pen stroke (Figure 4.5).

4.6 Task (ii): defect localisation

4.6.1 Method

Localisation reuses the trained classifiers. A **region classifier** is trained on the patch features of the *top five* proposals of every training image:

- the top proposal of a hole or pen-mark image is labelled with the image class;
- all proposals of defect-free images, and the lower-ranked proposals of defect images (mostly the static lens blemishes and texture specks), are labelled *background*. These act as **hard negatives** (5,235 training proposals: 694 defects, 4,541 background).

At inference, every proposal is scored, and the proposal with the highest $P(\text{defect})$ is the detected defect. Its connected-component bounding box, scaled back to the 2000×2000 image, is drawn

with the predicted class and confidence. The same decision also gives an image-level class (*multi-proposal* decision). The four classifier families were tuned for this task by image-level validation accuracy (Table 4.3).

A first version trained the region classifier only on top proposals. It was rejected after analysis because it labelled faint background specks on clean paper as holes: it had never seen such regions during training. The hard-negative training fixes this.

Table 4.3: Region classifier (multi-proposal decision) accuracy.

Classifier	Val. acc.	Test acc.
SVM (RBF, $C = 10$, $\gamma = 10^{-4}$)	**0.996**	**0.973**
Naive Bayes	0.969	0.969
Logistic Regression ($C = 0.01$)	0.987	0.964
Neural Network (256,64)	0.991	0.973

Because the multi-proposal SVM had the highest validation accuracy of all models (0.996), it was selected as the final model for both tasks. On the test set it recognises all 77 clean images correctly (no false alarms), 68 of 73 holes and 74 of 75 pen marks, giving 97.3% overall. The single-candidate classifiers of Table 4.2 are slightly better on test (up to 99.1%); the difference is 4 images and is within the noise of a 225-image test set.

4.6.2 Localisation results

No ground-truth boxes are provided, so localisation was evaluated in two ways:

- **Consistency.** For 146 of the 148 defective test images, the chosen proposal is also the darkest region of the image. In the other two, the region classifier correctly passed over a static blemish.
- **Visual inspection** of zoomed crops (Figure 4.4) and of all misclassified images (Figure 4.5). In every inspected case the box lies tightly on the real hole or pen mark, including the 6 images whose class was wrong. Pen-mark boxes cover the whole stroke, because the morphological closing merges its fragments.

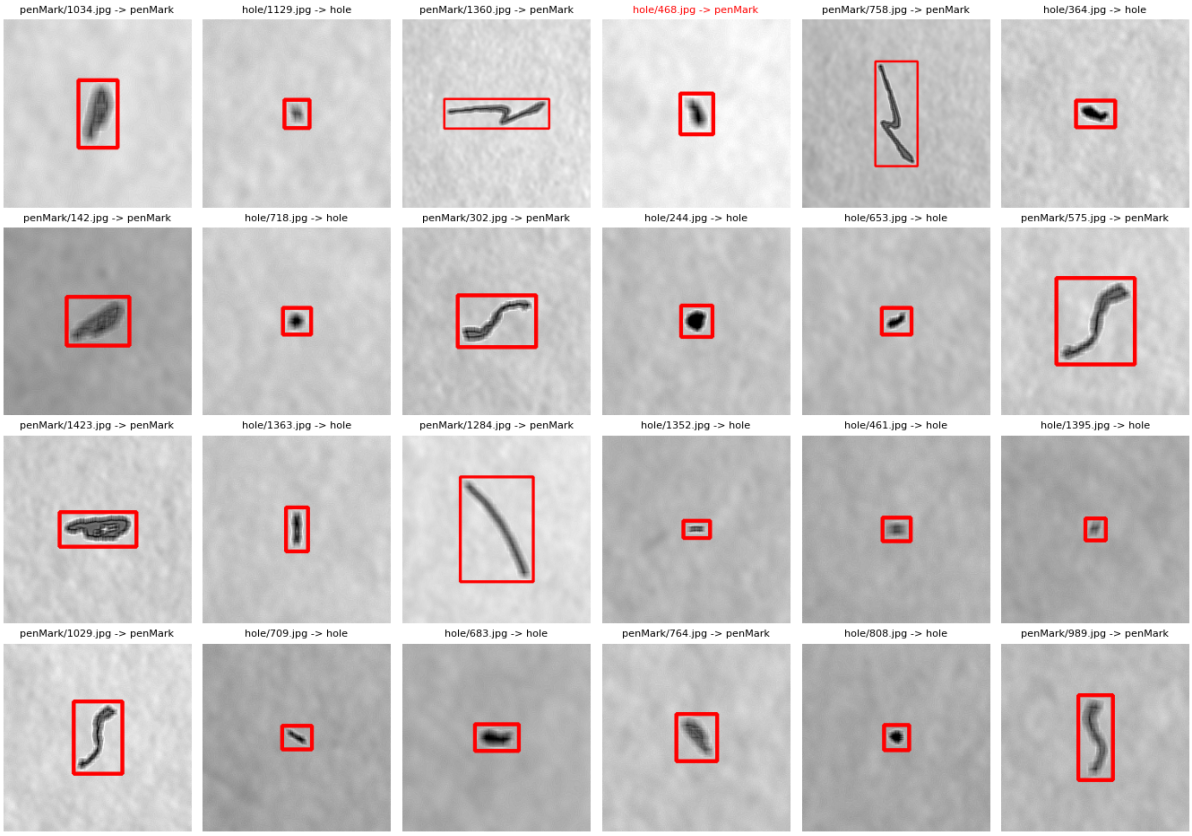


Figure 4.4: Detected bounding boxes on 24 random defective test images (zoomed on the box; titles show the true class and the prediction; red = wrong class).

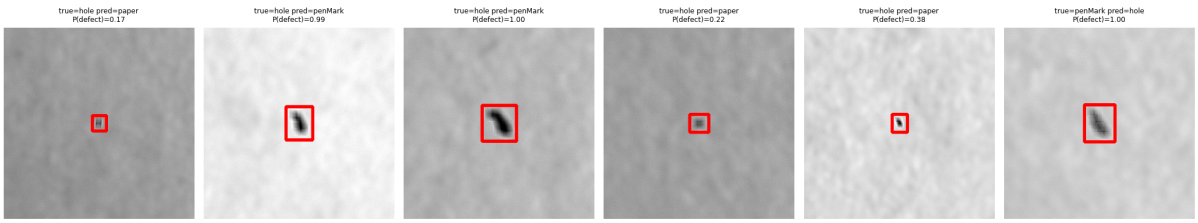


Figure 4.5: All misclassified test images. Each box is still on the defect; the errors are faint holes (predicted paper) and elongated holes resembling pen strokes.

4.7 Limitations

The method assumes a single dominant dark defect per image and dark defects on light paper. Bright defects (for example tears with back-lighting) would need a symmetric threshold. A quantitative IoU evaluation would require manually annotated boxes. Detecting very faint holes is the main remaining error source; higher-resolution processing around proposals could reduce it.

Chapter 5

Software Design and Ease of Use

5.1 Organisation of the code

Each question has a clear separation of concerns:

- a *feature module* (`audio_features.py`, `defect_features.py`) with pure functions for preprocessing and feature extraction, documented with docstrings;
- a *training script* that runs the whole experiment (split, features, tuning, evaluation, figures, model export) with one command;
- a *prediction script* (`predict.py`) that applies the saved model to new files.

All results (CSV tables, classification reports, figures, tuning logs) are written to each question's `results/` folder, and models are saved with `joblib` to `models/`.

5.2 How to run

```
pip install -r requirements.txt
python q1/q1_knn.py
python q2/q2_audio_sentiment.py --cutoff-study
python q2/q2_extras.py
python q3/q3_defect_detection.py
python q2/predict.py some_clip.wav
python q3/predict.py some_image.jpg        # writes annotated image to q3/predictions/
streamlit run app.py                       # interactive web demo
```

5.3 Interactive web application

`app.py` provides a browser interface to all three models. It accepts typed headlines, microphone recordings, uploaded WAV files and uploaded images, as well as examples from the test sets. Chapter 6 describes it in detail.

5.4 Public repository

The complete project (code, trained models, results, report) is published at `https://github.com/hbnasir23/ml-cep-ct317`. The audio and image datasets are not stored in the repository because of their size (1.2 GB and 1.6 GB); the README explains how to download them.

5.5 Reproducibility

Seeds are fixed (42), the expensive feature extraction is cached in `cache/`, and every number in this report is produced by the scripts. Two full runs of the Q3 pipeline gave identical results.

Chapter 6

Innovation: An Explainable Interactive Application

Beyond the required experiments, the project contributes an **interactive, explainable front-end**. Anyone can test the trained models with their own input, and each prediction comes with the evidence behind it. It is started with `streamlit run app.py` and has one tab per question.

6.1 Q1: explain a headline by its neighbours

The user types any headline. The app shows the predicted label, $P(\text{real})$ and how many of its words the model knows. Because KNN is an instance-based learner, the prediction can be explained *exactly*: the app lists the $k = 12$ nearest training headlines with their cosine distances and labels (Figure 6.1). This makes the effect of the cosine metric visible: neighbours share the same words regardless of length.

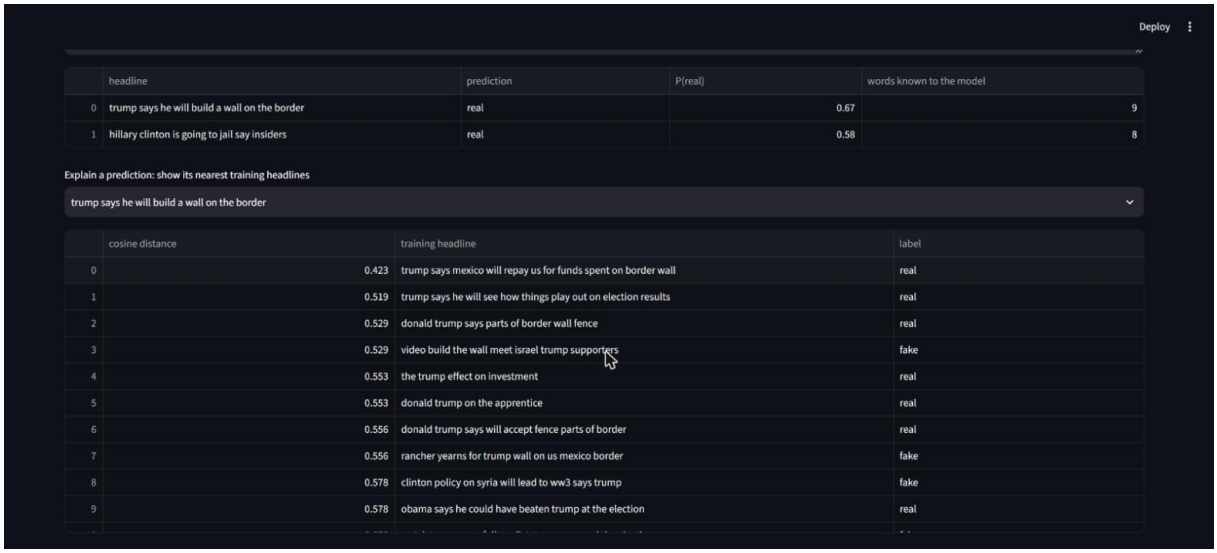


Figure 6.1: Q1 tab: prediction for user headlines and the 12 nearest training headlines that explain it.

6.2 Q2: live microphone input and "what the filter removed"

The user can **record their own voice in the browser**, upload a WAV file, or replay a clip from the unseen test speakers. The app shows the predicted emotion, the full probability distribution, and the spectrogram before and after the 4 kHz low-pass filter. Figure 6.2 is an

instructive case: the noisy recording contains a constant tone at about 7.5 kHz, which the filter removes completely. The model hesitates between *Happy* (0.45) and the true *Anger* (0.43), two high-arousal emotions it often confuses.

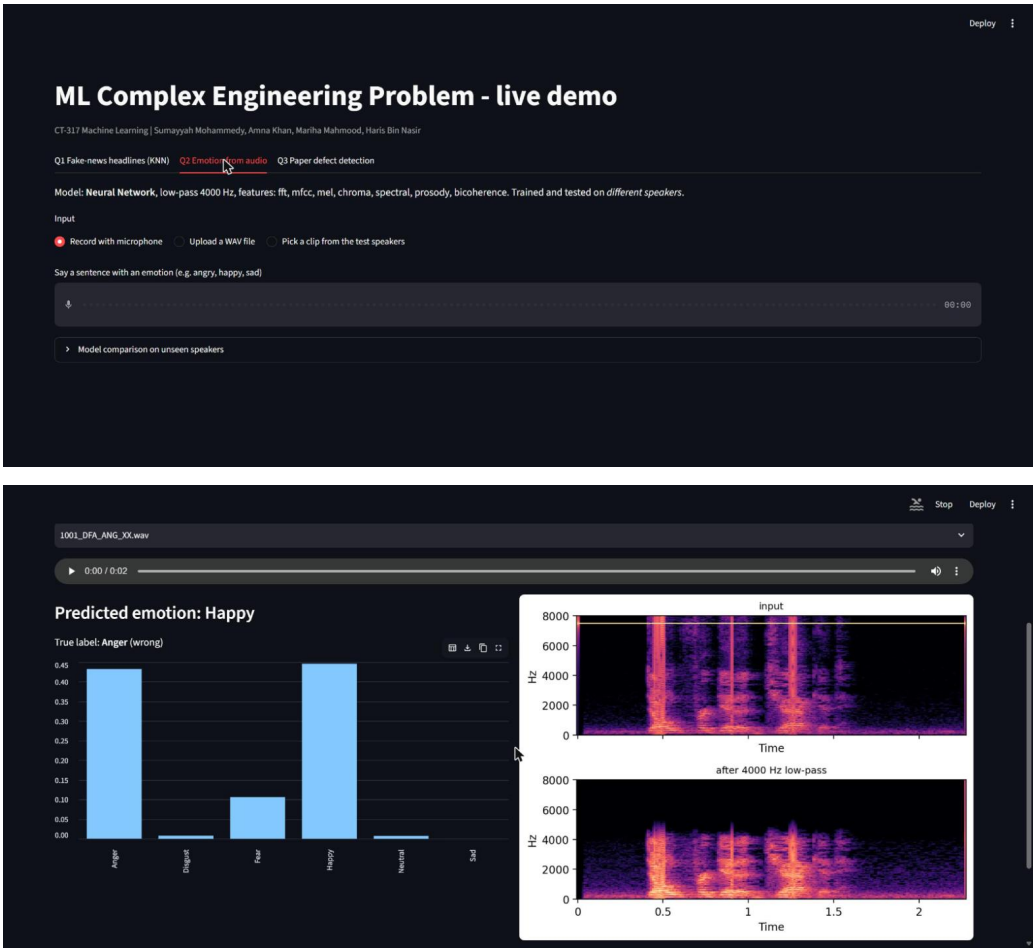


Figure 6.2: Q2 tab: microphone recording (top); prediction, probabilities and the spectrogram before and after filtering (bottom).

6.3 Q3: defect localisation with a look inside the pipeline

The user uploads a paper image or picks a test image. The app shows the class, its probabilities, the bounding box in original pixel coordinates and a zoom on the defect. An expandable panel shows *how the model sees the image*: the illumination-flattened z-map with all five candidate regions and, for every candidate, its darkness score, size and class probabilities from the region classifier (Figure 6.3). This makes it clear why a faint speck is rejected while the pen stroke is accepted.

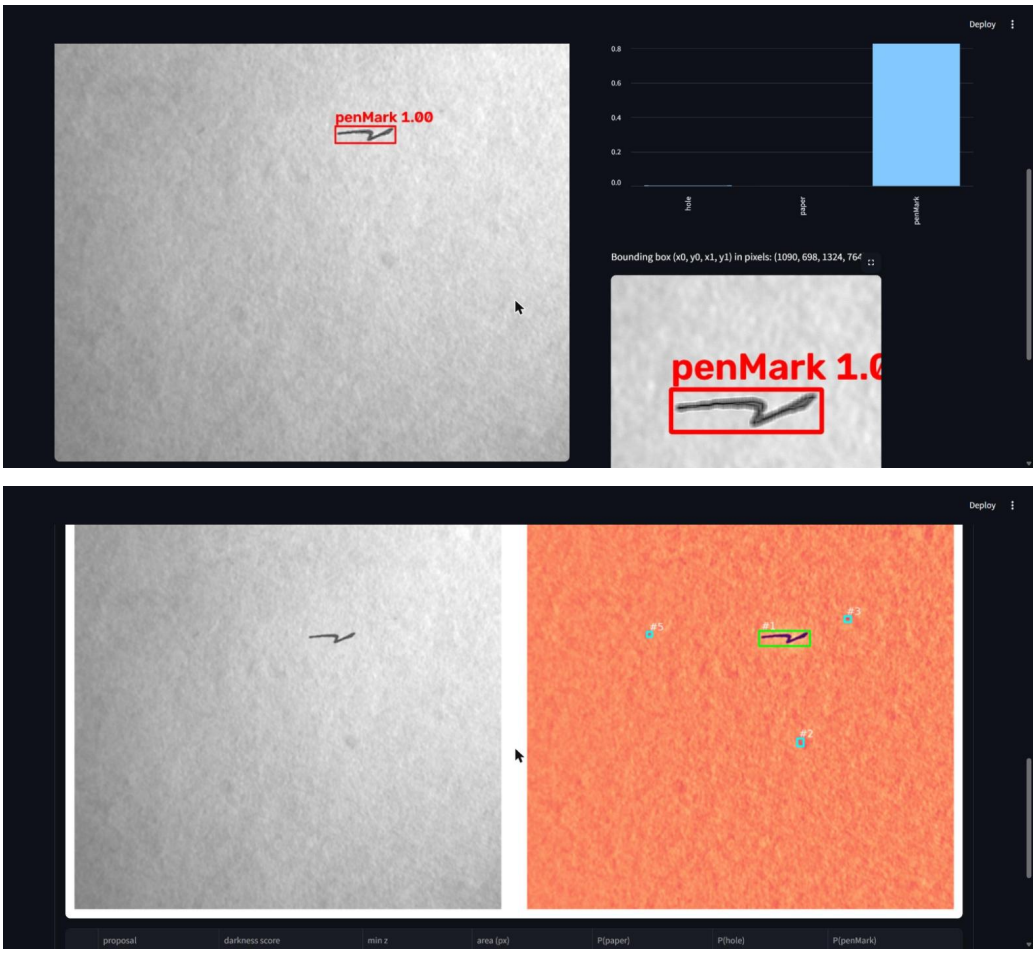


Figure 6.3: Q3 tab: detected pen mark with bounding box and zoom (top); z-map with the five scored proposals, where #1 (green) is the selected defect (bottom).

6.4 Methodological innovations

- **Speaker-independent evaluation and per-speaker normalisation (Q2).** Most simple tutorials split clips at random and report inflated accuracies. We evaluate on unseen voices and show that label-free per-speaker normalisation adds about 9 points (52.4%→61.3%).
- **Proposal-based classical detector with hard negatives (Q3).** Instead of describing the whole image, the pipeline isolates candidate objects (illumination flattening, robust z-score, morphology) and learns to reject static lens blemishes. The same classifier performs both classification and localisation.
- **Analysis beyond accuracy.** Feature ablations, a filter cut-off study, intensity-level analysis, error galleries and confusion matrices explain where and why the models fail.
- **Fully reproducible open-source project.** The code, trained models and results are public at `https://github.com/hbnasir23/ml-cep-ct317`. Every number in this report can be regenerated with one command per question.

Chapter 7

Team Contribution

Table 7.1: Individual contributions.

Member	Main responsibilities
Sumayyah Mohammedy (SE-23013)	Q1 end to end: `load_data` and `select_knn_model`, the Euclidean versus cosine experiments and their mathematical explanation. Q1 tab of the web app (nearest headline explanation). Compiled and edited the LaTeX report.
Amna Khan (SE-23039)	Q2 audio pipeline: literature review of speech-emotion features, speaker-independent data split, low-pass filter design and cut-off study, implementation of the 242 audio features including bicoherence. Feature-group ablation.
Mariha Mahmood (SE-23047)	Q2 modelling: tuning and evaluation of KNN, Logistic Regression, Naive Bayes, SVM and MLP, ensemble and per-speaker normalisation experiments. Q3 pre-processing (illumination flattening, z-map), candidate proposals and image feature extraction (histograms, spatial binning, HOG, LBP, GLCM, shape), data augmentation.
Haris Bin Nasir (SE-23084)	Q3 modelling: classifier comparison over three feature sets, hard-negative region classifier and bounding-box localisation, error analysis. Software integration: prediction scripts, Streamlit application (microphone input, explanation views), GitHub repository and reproducibility.

All members took part in weekly reviews of results, checked each other's code, and contributed to the discussion sections of the report.

Table 7.2: Responsibility matrix (**L** = lead, **S** = support).

Task	Sumayyah	Amna	Mariha	Haris
Q1 KNN classification	L	S		
Q2 data, filtering, features		L	S	
Q2 classifiers and analysis		S	L	
Q3 pre-processing and features			L	S
Q3 classifiers and localisation			S	L
Web application and repository	S			L
Report	L	S	S	S

Chapter 8

Conclusion and Future Work

- **Q1.** KNN on bag-of-words features classifies headlines with 63.9% test accuracy using Euclidean distance and **75.7%** using cosine distance. The cosine metric is length-invariant and measures word overlap, which suits sparse text vectors.
- **Q2.** With low-pass filtering, 242 literature-based audio features and a strict speaker-independent protocol, the SVM recognises six emotions with **52.4%** accuracy on unseen speakers. Per-speaker normalisation raises this to **61.3%**. MFCCs and prosodic features are the most informative.
- **Q3.** Illumination flattening, defect proposals and patch-level texture and shape features give **97–99%** three-class accuracy with all four classifiers. A hard-negative region classifier localises every inspected defect.

Future work: deep audio embeddings (wav2vec 2.0) and data augmentation for Q2; manually annotated boxes for an IoU evaluation and a sliding-window or CNN detector for multiple defects in Q3; TF-IDF weighting and linear models for Q1.

Bibliography

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

[2] B. McFee *et al.*, "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf.*, 2015.

[3] S. van der Walt *et al.*, "scikit-image: image processing in Python," *PeerJ*, 2:e453, 2014.

[4] C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[5] H. Cao, D. G. Cooper, M. K. Keutmann, R. C. Gur, A. Nenkova, R. Verma, "CREMA-D: Crowd-sourced emotional multimodal actors dataset," *IEEE Trans. Affective Computing*, vol. 5, no. 4, pp. 377–390, 2014.

[6] L. Rabiner, R. Schafer, *Theory and Applications of Digital Speech Processing*, Pearson, 2010.

[7] M. El Ayadi, M. S. Kamel, F. Karray, "Survey on speech emotion recognition: Features, classification schemes, and databases," *Pattern Recognition*, vol. 44, no. 3, pp. 572–587, 2011.

[8] B. Schuller, "Speech emotion recognition: Two decades in a nutshell, benchmarks, and ongoing trends," *Commun. ACM*, vol. 61, no. 5, pp. 90–99, 2018.

[9] E. A. AlBadawy, S. Lyu, H. Farid, "Detecting AI-synthesized speech using bispectral analysis," in *Proc. CVPR Workshops*, 2019.

[10] X. Xie, "A review of recent advances in surface defect detection using texture analysis techniques," *Electronic Letters on Computer Vision and Image Analysis*, vol. 7, no. 3, pp. 1–22, 2008.

[11] N. Neogi, D. K. Mohanta, P. K. Dutta, "Review of vision-based steel surface inspection systems," *EURASIP J. Image and Video Processing*, 2014:50, 2014.

[12] Udacity Self-Driving Car Nanodegree, "Vehicle detection: colour histograms, spatial binning and HOG features," course material, 2017.

[13] N. Dalal, B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. CVPR*, 2005.

[14] T. Ojala, M. Pietikäinen, T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE TPAMI*, vol. 24, no. 7, pp. 971–987, 2002.

[15] R. M. Haralick, K. Shanmugam, I. Dinstein, "Textural features for image classification," *IEEE Trans. SMC*, vol. 3, no. 6, pp. 610–621, 1973.

[16] M.-K. Hu, "Visual pattern recognition by moment invariants," *IRE Trans. Information Theory*, vol. 8, no. 2, pp. 179–187, 1962.

Appendix A

Requirement Checklist

Table A.1: CEP requirements and where they are addressed.

Requirement	Section / file
Q1(a) `load_data`: CountVectorizer, 70/15/15 split	§2.3.1, `q1/q1_knn.py`
Q1(b) `select_knn_model`, $k = 1..20$, plot, test accuracy	§2.4, Fig. 2.1, Tab. 2.1
Q1(c) cosine metric and explanation	§2.5
Q2 data selection and train/val/test sets	§3.3, Tab. 3.1
Q2 noise removal with low-pass filtering	§3.4, Fig. 3.1
Q2 literature review and feature list (FFT, MFCC, mel, chroma, bicoherence, spectral centroid/bandwidth/contrast)	§3.5, Tab. 3.2
Q2 KNN, LR, NB, SVM, NN evaluation	§3.6, Tab. 3.3
Q3 colour histogram, spatial binning, HOG + further features	§4.4, Tab. 4.1
Q3 data augmentation (optional)	§4.4
Q3 SVM, NB, LR, NN three-class classification	§4.5, Tab. 4.2
Q3 defect localisation with bounding boxes	§4.6, Fig. 4.4

Appendix B

Selected Hyper-parameter Grids

```
Q2  KNN   n_neighbors {3,5,9,15,21,31} x weights {uniform,distance} x metric {
    euclidean,cosine}
    LR    C {0.001,0.01,0.1,1,10}
    NB    var_smoothing {1e-9,1e-7,1e-5,1e-3,1e-2,1e-1}
    SVM   C {0.3,1,3,10,30} x gamma {scale,0.001,0.003,0.01}
    MLP   hidden {(128),(256),(256,128),(512,256)} x alpha {1e-3,1e-2,1e-1,1}
Q3  SVM   C {0.3,1,3,10,30} x gamma {scale,1e-4,3e-4}
    NB    var_smoothing {1e-9,1e-6,1e-3,1e-2,1e-1}
    LR    C {0.001,0.01,0.1,1}
    MLP   hidden {(128),(256),(256,64)} x alpha {1e-3,1e-2,1e-1,1}
```